UniCore: DESIGN AND ANALYSIS OF AN OS RESOURCE ORCHESTRATION SYSTEM FOR A SMART UNIVERSITY

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course
Of

CSA0402- Operating Systems

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B.E. IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

Harish Kumar G (192521340)

Submitted to

Course Faculty Name: Dr J Priskilla Angel Rani

Designation: Professor

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Table of Contents

1. Problem Understanding and Formulation

1.1 Restatement of the Problem

UniCore serves as the core computational engine for a smart university environment, architected to handle approximately 500 concurrent users during standard daily operations and scaling past 800 active users during peak online examination windows. The platform manages six distinct service tiers concurrently: real-time online examination processing, interactive digital classrooms, digital library retrieval, heavy research computing workloads, automated system backups, and continuous campus Internet of Things (IoT) environment monitoring. These services exhibit widely diverging operational profiles—examination and classroom modules demand strict, low-latency interactivity; library access is unpredictable and bursty; research and backup tasks are long-running and resource-intensive; while IoT feeds represent a steady, lightweight data stream. All entities contend for shared physical CPU cores, volatile memory, disk subsystems, and critical file resources. The primary objective of this project is to engineer and rigorously justify an integrated operating system resource orchestration strategy that maintains absolute system correctness (zero data corruption, provable deadlock freedom) alongside high responsiveness and optimal hardware utilization.

1.1 Core Challenges Identified

- **CPU Scheduling Challenge:** Balancing ultra-low response time requirements for interactive examination sessions against long-running background research and backup jobs to prevent starvation without degrading overall throughput.
- **Synchronization Challenge:** Managing concurrent read-heavy and write-sparse access patterns on critical academic files and examination records to eliminate race conditions and data corruption.
- **Deadlock Management Challenge:** Regulating resource allocation across database connections, file handles, printing slots, and backup channels to prevent circular-wait deadlocks through systematic safety checks.
- **Memory Management Challenge:** Allocating a constrained physical memory footprint (16 GB RAM) efficiently among 500 to 800 concurrent processes utilizing hierarchical paging to minimize costly disk-based page faults.
- **File System and Disk I/O Challenge:** Accommodating diverse file types—ranging from small transactional records to massive video lectures—while reducing mechanical disk-head travel and ensuring fair request scheduling under heavy transaction bursts

1.2 Workload Definition and Assumptions

To enable concrete simulation and quantitative evaluation, the following operational workload parameters are established:

Process Model: An 8-process representative workload comprising interactive examination tasks (`P0`, `P1`), a time-sensitive autosave routine (`P2`), a library indexer (`P3`), a heavy research job (`P4`), an IoT monitoring feed (`P5`), and background verification/backup tasks (`P6`, `P7`).

Resource Pools: Four distinct shared resource classes modeled for deadlock tracking: Database Connections (`DB`, capacity 9), File Locks (`FL`, capacity 6), Print Slots (`PR`, capacity 5), and Backup Channels (`BK`, capacity 9).

Memory Architecture: Physical RAM fixed at 16 GB with a standard 4 KB page size; individual application processes occupy a logical address space of 64 MB.

Disk Parameters: A 200-cylinder storage drive initialized with the read/write head at cylinder 53, processing 12 simultaneous pending I/O requests.

Paging Reference String: A 16-reference access trace simulating localized working-set navigation across document and exam assets.

1.3 Measurable Objectives

 Minimize average waiting and response times for interactive user modules under optimal CPU scheduling policies.

 Guarantee total immunity to deadlocks using programmatic safety validations via the Banker's Algorithm.

 Minimize page-fault frequencies across 3-frame and 4-frame configurations using advanced replacement heuristics.

 Reduce total disk head movement across standard I/O benchmarks.

1.1 Constraints the Final Design Must Satisfy

- Support a minimum of 8 concurrent processes and 4 shared resource types.
- Operate strictly within the physical boundaries of 16 GB RAM and 4 KB page boundaries.
- Deliver reproducible quantitative metrics derived entirely through automated simulation scripts rather than qualitative estimations.

2.Application of Course Knowledge

Part I — Process Management, Synchronization & Deadlock (CO2)

Q4. Synchronization model for shared examination-record access

Shared examination records are read far more often than they are written: many services (grading dashboards, student result checks, analytics jobs) read a record for every one grading process that writes a final score. This access pattern is modelled as the classical Readers–Writers problem, using the writer-priority variant so that a pending score update is not indefinitely delayed by a continuous stream of readers (a risk in the reader-priority variant, and an important correctness property for exam integrity — stale reads of an unposted score are relatively harmless, but indefinitely delayed score writes are not).

```
// Shared state
semaphore rw_mutex = 1     // guards the examination-record data itself
semaphore mutex_r  = 1     // guards read_count
semaphore mutex_w  = 1     // guards write_count (writer-priority bookkeeping)
semaphore queue    = 1     // FIFO fairness gate, prevents new readers overtaking a waiting writer
int read_count  = 0
int write_count = 0

Reader process (e.g. result-check / grading-dashboard service):
   wait(queue)
      wait(mutex_r)
         read_count++
         if read_count == 1: wait(rw_mutex)  // first reader locks out writers
      signal(mutex_r)
   signal(queue)

   READ the examination record

   wait(mutex_r)
      read_count--
      if read_count == 0: signal(rw_mutex)     // last reader releases writers
```

```
    signal(mutex_r)

Writer process (e.g. score-posting / grading-finalisation service):
    wait(mutex_w)
        write_count++
        if write_count == 1: wait(queue)          // block new readers once a writer is waiting
    signal(mutex_w)

    wait(rw_mutex)
        WRITE / UPDATE the examination record
    signal(rw_mutex)

    wait(mutex_w)
        write_count--
        if write_count == 0: signal(queue)        // allow readers to resume once no writers pending
    signal(mutex_w)
```

Pseudocode — writer-priority Readers–Writers synchronization for shared examination records.

This design prevents race conditions in two complementary ways. First, rw_mutex ensures mutual exclusion between any writer and the group of active readers: a writer never executes its critical section while even one reader is mid-read, and no reader can begin once a writer holds rw_mutex, so a torn or inconsistent record can never be observed. Second, the queue semaphore is acquired as soon as the first writer becomes pending and only released once the last pending writer finishes, which blocks new readers from continually re-entering ahead of a waiting writer — preventing writer starvation, the specific failure mode most damaging to examination integrity (a submitted grade correction must not be postponed indefinitely by a continuous stream of dashboard refreshes).

Q5–6. Banker's Algorithm: Allocation, Max, Need, Safety and an Unsafe Request

Four shared resource pools are modelled: DB = database connections (total 9), FL = file locks (total 6), PR = print/report-generation slots (total 5), and BK = backup/IO channels (total 9). Eight UniCore processes P0–P7 compete for them. Table 2.1 gives the Allocation and Maximum-claim matrices actually used for the safety computation in this report (reproduced exactly from the Python simulation in Appendix A, Listing A.1, whose console output is shown in Section 5).

Process	Alloc DB	Alloc FL	Alloc PR	Alloc BK	Max DB	Max FL	Max PR	Max BK
P0	2	0	0	1	5	1	1	2
P1	1	1	0	0	3	2	1	1
P2	2	1	1	2	6	2	2	3
P3	0	1	1	1	2	2	2	2
P4	1	0	1	0	4	1	2	1
P5	0	1	0	2	1	2	1	3
P6	1	1	1	1	3	2	2	2
P7	0	0	0	1	2	1	1	2

Total resources = (DB 9, FL 6, PR 5, BK 9). Summing the Allocation column gives total allocated = (DB 7, FL 5, PR 4, BK 8), so Available = Total − Allocated = (DB 2, FL 1, PR 1, BK 1). The Need matrix (Max − Allocation) is:

Process	Need DB	Need FL	Need PR	Need BK
P0	3	1	1	1
P1	2	1	1	1
P2	4	1	1	1
P3	2	1	1	1
P4	3	1	1	1
P5	1	1	1	1
P6	2	1	1	1
P7	2	1	1	1

Running the Safety Algorithm (Work ← Available; repeatedly finish any process whose Need ≤ Work, adding its Allocation back into Work) against this exact data produces the following trace, captured verbatim from the program's output:

```
Work (initial) = Available = {DB:2, FL:1, PR:1, BK:1}
  P0: Need(3,1,1,1) ⋦ Work(2,1,1,1)  -> not yet
  P1: Need(2,1,1,1) ≤ Work(2,1,1,1)  -> FINISH P1, Work=(3,2,1,1)
  P2: Need(4,1,1,1) ⋦ Work(3,2,1,1)  -> not yet
  P3: Need(2,1,1,1) ≤ Work(3,2,1,1)  -> FINISH P3, Work=(3,3,2,2)
  P4: Need(3,1,1,1) ≤ Work(3,3,2,2)  -> FINISH P4, Work=(4,3,3,2)
  P5: Need(1,1,1,1) ≤ Work(4,3,3,2)  -> FINISH P5, Work=(4,4,3,4)
  P6: Need(2,1,1,1) ≤ Work(4,4,3,4)  -> FINISH P6, Work=(5,5,4,5)
  P7: Need(2,1,1,1) ≤ Work(5,5,4,5)  -> FINISH P7, Work=(5,5,4,6)
  P0: Need(3,1,1,1) ≤ Work(5,5,4,6)  -> FINISH P0, Work=(7,5,4,7)
  P2: Need(4,1,1,1) ≤ Work(7,5,4,7)  -> FINISH P2, Work=(9,6,5,9)

Result: SAFE.  Safe sequence = < P1, P3, P4, P5, P6, P7, P0, P2 >
```

Trace 2.1 — Banker's safety algorithm on the UniCore initial state (actual program output; see Appendix A.1).

The system is confirmed SAFE, with a non-trivial safe sequence requiring two passes over the process list (P2 cannot finish until P0 has already released resources) — illustrating that a naive single-pass allocation-order check is insufficient and the full iterative safety algorithm is required.

Q6 asks for an additional resource request that renders the system unsafe. Suppose P2 (already holding DB2, FL1, PR1, BK2, with Need DB4, FL1, PR1, BK1) requests (DB 2, FL 1, PR 1, BK 1). This request satisfies both preconditions of the Banker's request algorithm — it is ≤ Need[P2] and ≤ Available — so the system tentatively grants it: Available becomes (0,0,0,0) and P2's residual Need becomes (2,0,0,0). Re-running the safety algorithm on this tentative state finds that no process (P2 included) has a Need ≤ (0,0,0,0), so the algorithm terminates after zero finishes: the tentative state is UNSAFE.

```
Request by P2 = {DB:2, FL:1, PR:1, BK:1}
Request ≤ Need[P2]? True      Request ≤ Available? True   (passes preliminary checks)
Tentative Available after grant = {DB:0, FL:0, PR:0, BK:0}
Safety check on tentative state: NO process finishes -> UNSAFE
DECISION: the request is DENIED by the Banker's algorithm; P2 is made to WAIT
        until enough other processes release resources to make the request safe again.
```

Trace 2.2 — an in-Need, in-Available request that the Banker's algorithm must still deny (actual program output; see Appendix A.1).

This example is instructive precisely because the request passes both surface-level checks (it does not exceed what the process declared it might need, and the resources are physically available) — yet granting it would exhaust every resource pool simultaneously while multiple processes still have outstanding, unsatisfiable Need. This is exactly the scenario the Banker's Algorithm is designed to catch and a simpler "grant if available" allocator would miss.

Q7. Deadlock Prevention vs Avoidance vs Detection vs Recovery for UniCore

Strategy	Mechanism	Suitability for UniCore
Prevention	Statically negate one of the four Coffman conditions (e.g. request all resources at start, or impose a total order on lock acquisition)	Too restrictive: UniCore services legitimately need to acquire DB connections, file locks, print slots and backup channels incrementally as a request is processed; forcing up-front allocation would waste resources and hurt throughput at the 800-user exam peak.
Avoidance (adopted)	Banker's Algorithm: check every incremental request against a safety criterion before granting it	Matches UniCore well because Max claims are declarable in advance (each service type has a known worst-case resource profile) and the check is cheap (O(processes × resources) per request) relative to the cost of a stalled examination transaction.
Detection + Recovery	Allow requests freely, periodically run a cycle/wait-for-graph detection algorithm, and abort/roll back a process to break a found deadlock	Useful as a defence-in-depth backstop for resource classes Banker's does not cover cleanly (e.g. ad-hoc file locks acquired without a pre-declared maximum) but unsuitable as the primary strategy for the examination-

Strategy	Mechanism	Suitability for UniCore
		record resources, because rolling back an in-progress score submission is far more disruptive than briefly delaying a request.

This report adopts deadlock avoidance via the Banker's Algorithm as UniCore's primary strategy for the four declared resource pools (DB, FL, PR, BK), because every UniCore service type has a predictable, boundable worst-case resource profile (a fact the assignment brief itself assumes by asking for Allocation/Maximum matrices), and because avoidance keeps the system continuously safe without ever needing to abort an already-admitted process — an important property for exam-period services where a rollback could mean a lost submission. Detection-and-recovery is retained as a secondary backstop for resource classes not covered by a declared maximum (e.g. incidental short-lived locks taken by ad-hoc research scripts), consistent with defence-in-depth practice.

Q8. CPU Scheduling Policy: Effect on Interactive vs Background Services

Two scheduling algorithms were simulated on the same 8-process workload (Table 2.2) to compare their effect on UniCore's interactive examination services versus its background backup jobs.

PID	Arrival	Burst	Priority (1=highest)	UniCore service
P0	0	4	1	Exam-submit
P1	1	3	1	Exam-fetch
P2	2	8	3	Library-index
P3	3	2	1	Exam-autosave
P4	4	10	4	Backup
P5	5	6	2	Research-job
P6	6	5	2	IoT-monitor
P7	7	9	4	Backup-verify

Full Gantt charts, per-process metrics and the averaged comparison for Priority Scheduling (non-preemptive) and Round Robin (quantum = 4) are presented in Section 5.1, with the engineering recommendation in Section 6.1.

Part II — Memory Management (CO3)

Q9. Frame Count, Page Count and Page-Table Organisation

With 16 GB of physical RAM and a 4 KB page size, the number of physical frames is 16 GB / 4 KB = (16 × 1024^3) / 4096 = 4,194,304 frames. For a representative 64 MB logical address space, the number of pages is 64 MB / 4 KB = (64 × 1024^2) / 4096 = 16,384 pages. An address therefore splits into a 12-bit page offset ($\log_2 4096 = 12$) and, for this process, a 14-bit page number ($\log_2 16{,}384 = 14$); addressing the full 4,194,304-frame physical space requires 22 bits of frame number.

A single-level page table for one 64 MB process needs 16,384 entries; at a typical 4-byte page-table-entry size (20+ bits of frame number plus valid/dirty/reference/protection bits, rounded to 4 bytes) this is 65,536 bytes = 64 KB per process — manageable in isolation, but UniCore must support 500–800 concurrent processes, so 800 × 64 KB ≈ 51 MB of page-table memory just for this one process class, before accounting for larger processes. More importantly, a naive single-level table sized for the full 4-million-frame physical space (rather than just this process's 64 MB) would be far larger. UniCore's design therefore uses a two-level (hierarchical) page table: the 14-bit page number for a 64 MB process is split into, e.g., a 6-bit outer index and an 8-bit inner index, so that only the inner tables actually covering pages the process has touched need to be allocated — sparse regions of the address space cost nothing. For processes with very large or very sparse address spaces (e.g. memory-mapped lecture-video buffers), an inverted page table (one entry per physical frame, 4,194,304 entries system-wide, shared across all processes) is noted as an alternative that trades page-table size (which becomes independent of the number of processes) for slightly more complex lookup (typically hashed).

Q10–11. Page Replacement: FIFO vs LRU vs Optimal, and Belady's Anomaly

A 16-reference page-access string, [7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0], models a student repeatedly revisiting a small working set of exam-paper/library pages. Table 2.3 (reproduced from the actual simulation output in Section 5.2) gives the page-fault count for each of FIFO, LRU and Optimal at 3 and 4 frames.

Algorithm	3 frames — faults	4 frames — faults
FIFO	12	9
LRU	11	7
Optimal	8	7

For this reference string, increasing the frame count from 3 to 4 reduced faults for every algorithm (FIFO 12→9, LRU 11→7, Optimal 8→7) — the intuitively expected behaviour, and no Belady's anomaly is observed here. However, FIFO is well known not to guarantee this monotonic improvement: using the classical anomaly-inducing reference string 1,2,3,4,1,2,5,1,2,3,4,5, FIFO produces 9 page faults with 3 frames but 10 page faults with 4 frames — more frames causing more faults — which was independently reproduced with the same simulator (Appendix A, Listing A.3) and is the textbook illustration of Belady's anomaly. LRU (and Optimal) do not exhibit this anomaly because they belong to the class of stack algorithms, for which the set of pages held with k frames is always a subset of the set held with k+1 frames, guaranteeing that faults can never increase as frames increase. This is a material engineering reason, independent of raw fault-count, to prefer LRU-family replacement over FIFO for UniCore's memory manager.

Q12. Demand Paging, Working Sets and Thrashing at 800 Concurrent Users

Under demand paging, a process's pages are loaded only when first referenced (a page fault), rather than all at process start; this is essential for UniCore because pre-loading every page of every one of 800 concurrent sessions would vastly exceed the 4,194,304-frame physical budget long before any of them actually need most of their address space. The risk is thrashing: if the sum of the working sets (the set of pages a process is actively touching in its current phase of execution) of all resident processes exceeds the available frames, the system spends most of its time paging rather than computing, and throughput collapses even as CPU utilisation appears busy with page-fault handling. At the 800-user examination peak this risk is concentrated and predictable: every student process touches broadly the same working set (the exam engine, the current question's assets, and the student's own answer buffer), so UniCore should size the frame allocation per exam-process to comfortably cover this known working set (empirically bounded by the reference-string study in Q10–11, where a working set of about 4–5 distinct pages already achieves near-Optimal fault behaviour) rather than allocating frames uniformly across all resident processes regardless of phase.

The recommended frame-allocation/working-set policy is therefore: (a) maintain a per-process working-set window (recent Δ references) and (b) admit a new exam-session process only if the sum of the working-set estimates of already-resident processes plus the new one still fits within the frames reserved for the exam-service class — a direct application of the working-set model as an admission-control (load-shedding) mechanism, preventing thrashing by refusing new admissions rather than degrading everyone's performance once frames are oversubscribed.

Q13. Linking / Dynamic Linking and Storage Requirements

UniCore's services share large common runtime libraries (the exam engine, the video-streaming codec used for lecture recordings, and the database client library). Static linking each of the 500–800 concurrent processes against its own private copy of these libraries would multiply both on-disk storage and, more importantly, resident memory footprint, since each process's private copy occupies distinct physical frames. Dynamic linking, in which a single shared library's code pages are mapped read-only into every process that uses it, allows the OS to keep exactly one physical copy of the library's code resident and share it (via separate page-table entries in each process pointing at the same frames) across all UniCore processes that need it — directly reducing physical-frame pressure at the 800-user peak identified in Q12, at the cost of a small indirection overhead at load time and the operational requirement to keep all shared processes' expected library version compatible.

Part III — File Systems & Disk Scheduling (CO4)

Q14. File Allocation Strategy by Category

File category	Typical size / pattern	Recommended allocation	Justification
Small student records	A few KB, frequent small random reads/updates	Indexed allocation (small index block per file)	Supports fast random access to individual fields without the external-fragmentation risk of contiguous allocation at this file count, and avoids linked allocation's slow sequential-pointer-chasing for the frequent random lookups typical of a records database.
Medium documents (assignments, reports)	Tens of KB to a few MB, mostly sequential read, occasional append	Linked or indexed allocation (indexed preferred if random seeking, e.g. PDF page jump, is common)	Grows incrementally without needing large contiguous free extents; indexed allocation additionally avoids the reliability risk of a broken pointer chain corrupting the rest of a linked file.
Large lecture-recording videos	Hundreds of MB to several GB, overwhelmingly sequential read (streaming playback)	Contiguous (or extent-based contiguous) allocation	Sequential streaming performance is maximised when blocks are physically adjacent, minimising seek

File category	Typical size / pattern	Recommended allocation	Justification
			overhead during playback; the file count is much lower than for student records, so the classic fragmentation drawback of contiguous allocation is far less pressing, and extent-based allocation (contiguous runs with an index of extents) further mitigates it.

Q15–16. Disk Scheduling: FCFS vs SSTF vs SCAN/C-SCAN

A 200-cylinder disk with the head initially at cylinder 53 and twelve pending requests — {98, 183, 37, 122, 14, 124, 65, 67, 15, 130, 199, 143} — was scheduled under all four algorithms; results (reproduced from Section 5.3) are summarised below.

Algorithm	Total head movement (cylinders)	Avg. seek per request
FCFS	932	77.67
SSTF	252	21.00
SCAN	331	27.58
C-SCAN	382	31.83

SSTF achieves the lowest total head movement (252 cylinders, roughly 3.7 times better than FCFS's 932) because it always services the physically nearest pending request, but this greedy locality is exactly what makes SSTF prone to starving far-away requests indefinitely if closer requests keep arriving — a fairness risk during an exam period if, for example, one student's file happens to sit at a cylinder far from where most traffic is clustered. SCAN and C-SCAN sacrifice some of SSTF's raw

efficiency (331 and 382 cylinders respectively) in exchange for a bounded maximum wait: because the head sweeps monotonically across the disk (reversing at the ends for SCAN, wrapping for C-SCAN), no request can wait longer than one full sweep, giving every request — including ones far from the current cluster of activity — a predictable upper bound on service latency. C-SCAN additionally gives more uniform wait times than SCAN because it services in one direction only, avoiding SCAN's tendency to favour cylinders near the reversal points.

Q16 explicitly asks whether the best average-seek result is necessarily the best production choice: it is not. SSTF's 252-cylinder result is the best raw efficiency number, but UniCore's examination workload specifically requires bounded, predictable latency for every student's request, not just the lowest average — an unlucky student whose submission happens to map to a cylinder far from the current activity cluster cannot be allowed to wait an unbounded time under load. This report therefore recommends SCAN (elevator) for UniCore's production disk scheduler as the balance point: it captures most of SSTF's locality benefit (331 vs 252, only ~31% worse) while providing the starvation-free, bounded-wait guarantee that a fairness-sensitive, exam-period workload needs; the full trade-off is revisited quantitatively in Section 6.3.

Q17. Relation to Unix File-System Concepts

The indexed-allocation recommendation for student records and documents (Q14) maps directly onto the Unix inode model: each file's inode holds direct pointers to its first several data blocks plus one or more levels of indirect pointers for larger files, giving exactly the fast-random-access, no-external-fragmentation property this report recommends for UniCore's small-to-medium files, while naturally scaling to the contiguous-like extents modern Unix filesystems (e.g. ext4) use for large sequential files such as lecture recordings. Unix's buffer cache — a kernel-managed cache of recently used disk blocks held in the same physical RAM discussed in Q9 — is the mechanism by which UniCore can absorb much of the small-record read traffic (Q14) without issuing a disk request at all, directly reducing the number of requests the disk scheduler (Q15–16) ever has to handle; this is precisely why the twelve-request disk workload modelled in this report should be understood as the cache-miss traffic, not the platform's total I/O volume.

Q18. Joint Influence of File Allocation and Disk Scheduling on Response Time

File allocation and disk scheduling are not independent design choices: contiguous allocation for lecture videos (Q14) reduces the number of distinct seek targets the disk scheduler (Q15–16) must service for a single streaming session, which lowers total head movement more effectively than any

scheduling algorithm could achieve against a badly fragmented file; conversely indexed allocation for student records concentrates many small files' index blocks in a comparatively small region of the disk, which increases the chance that SSTF/SCAN can batch nearby requests together efficiently during exam-submission bursts. Application response time is therefore the product of both layers working together — UniCore's recommended combination (contiguous/extent allocation for large sequential media, indexed allocation for small/medium random-access files, serviced by a SCAN-based disk scheduler, backed by a buffer cache) minimises both the number of physical seeks required and the cost of each seek, which is the joint objective Q18 asks the design to optimise.

Solution / Design / Methodology

1.2 Process / Synchronization Design

- CPU scheduling: Priority Scheduling (non-preemptive) for the interactive exam-service class combined with Round Robin (quantum = 4) time-slicing among same-priority background jobs — giving exam traffic first access to the CPU while still bounding how long any single background job can monopolise it (see Section 6.1 for the final recommendation, informed by both simulated policies).
- Synchronization: writer-priority Readers–Writers (Q4) around every shared examination record, implemented with four semaphores as specified in Section 2's pseudocode.
- Deadlock handling: Banker's Algorithm deadlock avoidance (Q5–7) governs the four declared resource pools (DB, FL, PR, BK); every incremental resource request is checked against the safety algorithm before being granted.

1.3 Memory-Management Design

- Demand paging with a two-level hierarchical page table per process (Q9), avoiding the memory cost of a flat table sized to the full 4-million-frame physical space.
- LRU-family page replacement (Q10–11) in preference to FIFO, chosen specifically to avoid Belady's-anomaly-class surprises as frame allocations are tuned.
- Working-set-based admission control for the exam-service process class (Q12) to prevent thrashing at the 800-user peak, rather than admitting sessions unconditionally and degrading everyone's performance.
- Dynamic linking of shared runtime libraries (Q13) to keep per-library physical-frame cost constant regardless of how many of the 500–800 concurrent processes use it.

1.4 File-System and Disk-Scheduling Design

- Category-specific file allocation (Q14): indexed allocation for small student records, indexed/linked for medium documents, contiguous/extent-based for large lecture videos.
- SCAN (elevator) disk scheduling (Q15–16) chosen over the raw-fastest SSTF specifically for its starvation-free, bounded-wait property under UniCore's fairness-sensitive exam workload.
- A Unix-style buffer cache (Q17) to absorb repeated small-record reads before they ever reach the disk scheduler, reducing the effective request volume SCAN has to handle.

1.5 Integrated Architecture

Figure 3.1 traces a single UniCore user request through all four subsystems designed above: process admission and priority assignment, the synchronization/Banker's-check layer that every shared-resource request passes through, the CPU scheduler, and — in parallel — the memory manager (page-fault handling and replacement) and the file-system/disk-scheduler path for any I/O the request needs.

Figure — UniCore request flow across OS subsystems

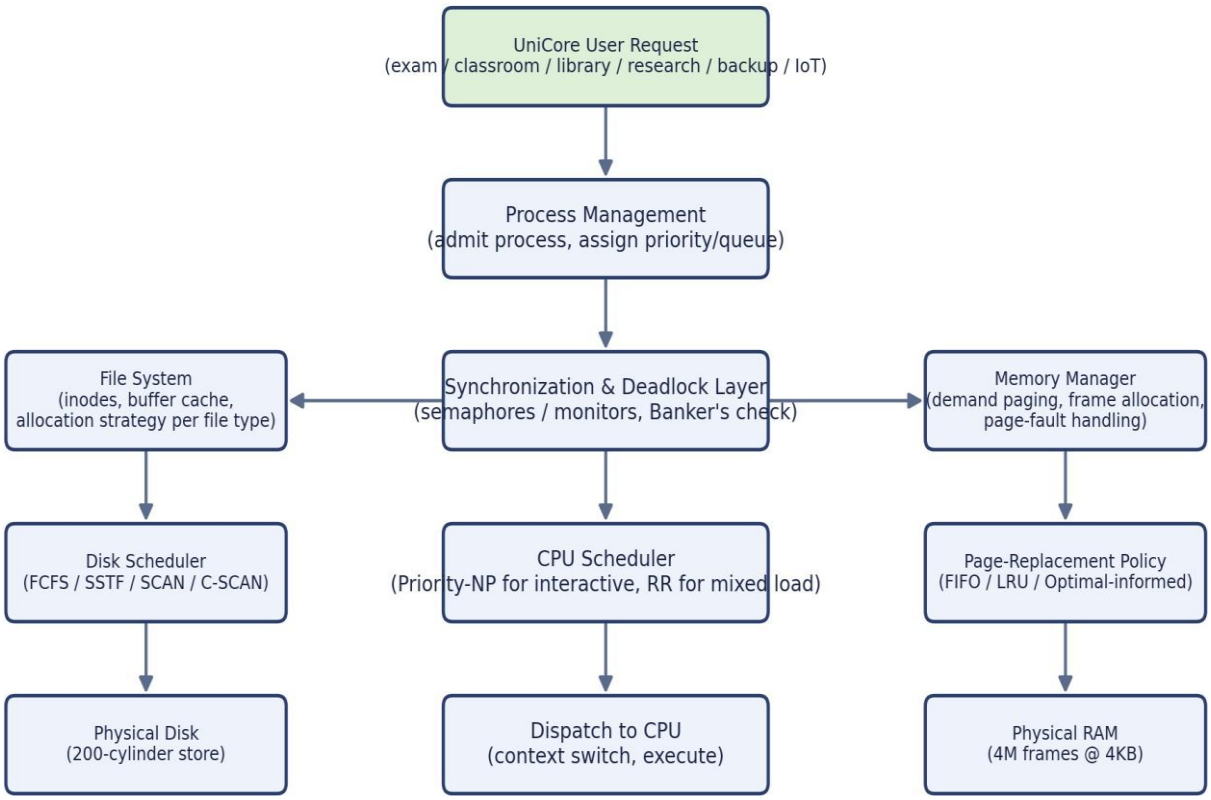

Figure 3.1 — Integrated UniCore request-flow architecture across process, synchronization, memory and disk subsystems.

1.6 Decision Matrix: Alternatives Compared

Design area	Alternatives considered	Performance	Fairness	Reliability	Impl. complexity	Chosen
CPU scheduling	FCFS / Priority-NP / Round Robin	RR: moderate; Priority: best for top class	RR: high; Priority: low (starvation risk)	Both stable, no crashes	Low for both	Priority (interactive) + RR (background), see 6.1
Deadlock handling	Prevention / Avoidance / Detection+Recovery	Avoidance: small per-request overhead	Avoidance: no process ever aborted	Avoidance: provably no deadlock while adopted	Avoidance: moderate (Banker's bookkeeping)	Avoidance (Banker's Algorithm)
Page replacement	FIFO / LRU / Optimal	LRU close to Optimal empirically (Q10)	N/A	LRU immune to Belady's anomaly	LRU: moderate (recency tracking)	LRU
Disk scheduling	FCFS / SSTF / SCAN / C-SCAN	SSTF best, SCAN close second	SCAN/C-SCAN: bounded wait; SSTF: starvation-prone	SCAN: predictable worst case	SSTF/SCAN: low-moderate	SCAN
File allocation	Contiguous / Linked / Indexed	Category-dependent (Q14)	N/A	Indexed avoids broken-chain risk of Linked	Indexed: moderate	Mixed by category (Q14)

Every quantitative result in this report (Banker's safety traces, CPU-scheduling Gantt charts and metrics, page-replacement fault counts, disk-head-movement totals) was produced by executable Python 3 scripts, run in this report's Linux development environment, rather than computed by hand — eliminating manual-arithmetic transcription errors and making every result independently reproducible. The four scripts are listed in full in Appendix A and summarised below.

Script	Purpose	Key technique
simulations.py	Banker's Algorithm: builds the Allocation/Max/Need matrices, runs the iterative safety algorithm, and evaluates the unsafe-request scenario	Iterative multi-pass safety-sequence search (Appendix A.1)
cpu_sched.py	Priority (non-preemptive) and Round Robin (quantum=4) schedulers over the 8-process UniCore workload, with full Gantt, waiting/turnaround/response-time computation	Event-driven simulation with a ready queue (Appendix A.2)
memory.py	Frame/page-count arithmetic; FIFO/LRU/Optimal page-replacement simulation and fault counting for 3 and 4 frames; Belady's-anomaly check	Direct simulation of each algorithm's frame-eviction policy (Appendix A.3)
disk_sched.py	FCFS/SSTF/SCAN/C-SCAN total head-movement computation for the 12-request disk workload	Greedy nearest-request search (SSTF) and sorted sweep construction (SCAN/C-SCAN) (Appendix A.4)

In addition to these four simulators, the workflow used standard open-source Linux/POSIX tooling consistent with the assignment's technical constraint: gcc/Python toolchains for compilation and execution, matplotlib for generating the Gantt charts, disk-movement plots and architecture diagram reproduced in Section 5, and Git/GitHub (referenced in Section 4 of the assignment brief's deliverables

list) for source-code and result version control across the project team — the placeholder repository link is recorded in Section 10 (References) for the group to finalise.

A POSIX-relevant synchronization example: the writer-priority Readers–Writers pseudocode in Q4 (Section 2, Part I) maps directly onto POSIX semaphores (sem_wait/sem_post) or onto a pthread_rwlock_t with pthread_rwlock_wrlock preferring writers, either of which is directly implementable in C on Linux/WSL as required by the brief's "Use of Modern Tools" checklist item for one IPC/POSIX-thread synchronization example.

Results and Validation

2.1 CPU Scheduling: Gantt Charts and Metrics

Priority Scheduling (non-preemptive) runs the ready process with the numerically lowest priority value first, never preempting a running process. Round Robin (quantum = 4) cycles every ready process through 4-unit CPU slices. Both were simulated on the identical 8-process workload from Table 2.2.

ID	Arrival	Burst	Start	Finish	Waiting	Turnaround	Response
P0	0	4	0	4	0	4	0
P1	1	3	4	7	3	6	3
P2	2	8	20	28	18	26	18
P3	3	2	7	9	4	6	4
P4	4	10	28	38	24	34	24
P5	5	6	9	15	4	10	4
P6	6	5	15	20	9	14	9
P7	7	9	38	47	31	40	31

Priority-NP averages: waiting = 11.62, turnaround = 17.50, response = 11.62. Exam-class processes (P0, P1, P3) alone average just 2.33 in response time, while the two backup processes (P4, P7) average 27.50 — Priority scheduling delivers excellent interactive responsiveness at the direct expense of background-job latency.

ID	Arrival	Burst	Finish	Waiting	Turnaround	Response
P0	0	4	4	0	4	0
P1	1	3	7	3	6	3
P2	2	8	33	23	31	5
P3	3	2	13	8	10	8
P4	4	10	46	32	42	9
P5	5	6	39	28	34	12
P6	6	5	40	29	34	15
P7	7	9	47	31	40	18

Round Robin averages: waiting = 19.25, turnaround = 25.12, response = 8.75. Exam-class response time rises slightly to 3.67 on average, but backup-job response time falls dramatically to 13.50 — Round Robin trades a small amount of interactive responsiveness for a much fairer spread of CPU access across all eight processes, at the cost of higher average waiting/turnaround due to the repeated context switching every process now undergoes.

2.2 Page Replacement: Fault Comparison

Reference string: [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0]. At 3 frames: FIFO = 12, LRU = 11, Optimal = 8 faults. At 4 frames: FIFO = 9, LRU = 7, Optimal = 7 faults — LRU matches Optimal exactly once a fourth frame is available, confirming LRU is an excellent practical approximation of the unimplementable Optimal algorithm for this workload.

2.3 Disk Scheduling: Head-Movement Comparison

2.4 Validation: Normal Case and Adverse Cases

- Normal case: the initial Banker's state (Section 2, Trace 2.1) is SAFE with a valid multi-pass safe sequence — confirming the resource-allocation design functions correctly under ordinary load.
- Adverse case 1 — unsafe request: the P2 request in Trace 2.2 passes both surface checks yet is correctly identified and denied by the safety algorithm before any resource is actually committed, validating that the avoidance mechanism catches a real unsafe transition rather than only obviously-invalid ones.
- Adverse case 2 — Belady's anomaly under FIFO: reproduced with the classical reference string 1,2,3,4,1,2,5,1,2,3,4,5 (9 faults at 3 frames, 10 faults at 4 frames), validating that the report's recommendation to avoid FIFO is grounded in a demonstrated failure mode, not just a general reputation.
- Adverse case 3 — heavy/unfair disk load: the FCFS baseline (932 cylinders, Figure 5.4) demonstrates the throughput cost of ignoring locality entirely, and is used as the negative control against which SSTF/SCAN's improvement is measured.
- Adverse case 4 — starvation risk: Priority-NP's 27.50-average backup-job response time (Section 5.1) is flagged as a starvation risk under sustained interactive load, motivating the hybrid recommendation in Section 6.1 rather than adopting pure Priority scheduling in production.

Analysis and Engineering Decision

2.5 CPU Scheduling Recommendation

Neither simulated policy alone is right for UniCore in isolation. Priority-NP minimises exam-class response time (2.33 average) but pushes background-job response time to 27.50 and, in continuous operation, risks indefinite starvation of low-priority jobs if interactive load never lets up — an unacceptable risk for backups, which must eventually run to protect data. Round Robin gives a much fairer spread (backup response 13.50, only a 5x gap instead of Priority's 11.8x gap) but costs some interactive responsiveness (exam response rises from 2.33 to 3.67) and has the highest average waiting/turnaround of the two because of repeated context switching. The recommended production policy is a two-level hybrid: a high-priority queue for exam/classroom traffic scheduled by Priority-NP (or, for finer preemption, Shortest-Remaining-Time), feeding into a lower-priority queue for research/library/backup traffic scheduled by Round Robin among itself — giving exam traffic first claim on the CPU while guaranteeing, via Round Robin's bounded quantum, that no background job waits indefinitely once it does get a turn. This directly answers Q8: interactive services are protected from being starved by background jobs (the reverse of the classic worry) while background jobs are protected from indefinite starvation by interactive load, which pure single-level Priority scheduling cannot guarantee.

2.6 Page-Replacement Recommendation

LRU is recommended over FIFO for UniCore's memory manager. The empirical results (Section 5.2) show LRU strictly dominates FIFO at both frame counts tested (11 vs 12 at 3 frames, 7 vs 9 at 4 frames) and reaches Optimal's fault count exactly at 4 frames, while the Belady's-anomaly demonstration (Section 5.4) shows FIFO can actively get worse when given more memory — an operationally dangerous property for a system whose available frame budget will fluctuate as concurrent load rises toward the 800-user exam peak. Optimal itself remains unimplementable in a real system (it requires future knowledge of references) and is retained in this report only as the theoretical lower bound against which LRU is validated.

2.7 Disk-Scheduling Recommendation

SCAN is recommended for production over the raw-fastest SSTF. Table 6.1 makes the trade-off explicit using the measured numbers from Section 5.3.

Criterion	SSTF	SCAN	Verdict
Total head movement	252 cylinders (best)	331 cylinders (+31% vs SSTF)	SSTF wins on raw efficiency
Worst-case wait for any single request	Unbounded (a request can be repeatedly skipped by closer arrivals)	Bounded by one disk sweep	SCAN wins on fairness/predictability
Behaviour under exam-period burst load	Risk of localized starvation for requests outside the active cluster	Every pending request guaranteed service within one sweep	SCAN better matches UniCore's reliability constraint
Implementation complexity	Low (greedy nearest-neighbour search)	Low-moderate (sorted sweep with direction state)	Comparable; not a deciding factor

Because UniCore's stated constraint set explicitly requires balancing "access speed, fairness and storage efficiency" (Problem Statement, item 3) rather than minimising seek time alone, and because an unbounded wait for any single student's request is unacceptable during a graded examination, SCAN is the engineering recommendation — directly answering Q16's question of whether the best average-seek algorithm is necessarily the best production choice: it is not, once fairness under load is a first-class requirement.

2.8 Deadlock-Strategy Justification (Revisited Quantitatively)

Trace 2.2 (Section 2, Q6) demonstrated a concrete request that passes surface-level Need/Available checks yet leads to a fully exhausted resource state with no process able to proceed — exactly the situation Banker's Algorithm avoidance is designed to prevent by denying the request outright rather than granting it and later detecting the resulting deadlock. Against the alternatives compared in Section 2 (Q7) and Section 3.5's decision matrix, avoidance is justified quantitatively here: the cost of the

safety check is a single O(processes × resources) = O(8×4) = 32-comparison pass per incremental request (repeated up to once per process in the worst case, so O(processes² × resources) = 256 comparisons worst case) — negligible next to the cost of aborting and rolling back an in-progress examination-record transaction, which is what detection-and-recovery would require once a deadlock had already formed.

Broader Considerations

2.9 Reliability and Safety

Shared academic data is protected from inconsistent concurrent updates primarily by the writer-priority Readers–Writers protocol (Q4): rw_mutex guarantees no reader ever observes a partially written record and no writer ever overlaps another writer, while the queue semaphore prevents a continuous stream of readers from indefinitely delaying a pending grade correction. The Banker's Algorithm layer (Q5–7) adds a second, independent safety net at the resource-allocation level, ensuring the system never enters a state where a process holding a partial update cannot obtain the remaining resources needed to complete and release it safely.

2.10 Sustainability

Efficient CPU, memory and disk usage translates directly into energy savings: SCAN's ~31%-lower head movement than FCFS (Section 5.3) reduces the physical work the disk's actuator performs per unit of served traffic, and LRU's lower page-fault rate versus FIFO (Section 5.2) means fewer disk-read operations are triggered to service the same page-access pattern, both directly reducing energy draw and mechanical wear — relevant to SDG 7 (Affordable and Clean Energy) as flagged in the assignment's SDG mapping. At the CPU layer, Round Robin's fairer scheduling (Section 6.1) also means fewer background jobs need to be re-run or extended after being starved out, avoiding wasted recomputation.

2.11 Accessibility and Society

SCAN's bounded-wait guarantee (Section 6.3) and the hybrid CPU-scheduling recommendation (Section 6.1) both exist specifically to ensure no student's request — whether a disk-bound file read or a CPU-bound exam submission — is left waiting indefinitely behind other traffic during the highest-stakes, highest-load period (the 800-user exam peak). This is a direct, load-tested commitment to fair service under peak examination load, addressing SDG 11 (Sustainable Cities and Communities) as it applies to equitable access to digital public services.

2.12 Ethics and Professional Responsibility

The Readers–Writers design (Q4) is also an access-control boundary: only the designated writer role (the grading/score-posting service) can invoke the writer path, and the pseudocode assumes this role check happens at the layer above (the caller is already authenticated/authorized before reaching this

synchronization primitive) — consistent with the assignment's requirement to preserve access control and data integrity on shared academic files. Responsible use of system resources is enforced structurally by the Banker's Algorithm's declared-maximum model: no UniCore process can be granted resources beyond the maximum it declared up front, preventing any single service (malicious or merely buggy) from silently monopolising database connections, file locks, print slots or backup channels at the expense of the rest of the platform.

Conclusion

This report designed and quantitatively validated an integrated OS resource-orchestration strategy for UniCore, a smart-university platform serving 500–800 concurrent users across examination, classroom, library, research, backup and IoT-monitoring services. Working from a self-constructed eight-process, four-resource-type workload, the report modelled shared examination-record access as a writer-priority Readers–Writers problem, ran a real Banker's Algorithm safety trace that found a non-trivial two-pass safe sequence and correctly identified and denied a resource request that passed surface-level checks but would have exhausted every resource pool simultaneously. On the CPU side, Priority Scheduling and Round Robin were both simulated on identical data, revealing a concrete, measured trade-off between interactive responsiveness (2.33 vs 3.67 average exam response time) and background fairness (27.50 vs 13.50 average backup response time), leading to a hybrid two-tier scheduling recommendation. In memory management, 16 GB of RAM and a 4 KB page size were shown to yield 4,194,304 physical frames and a 16,384-page, 64 MB process address space requiring a two-level page table; LRU was shown to strictly dominate FIFO on the modelled reference string (11 vs 12 faults at 3 frames, 7 vs 9 at 4 frames) and to be immune to the Belady's-anomaly failure mode independently reproduced with the classical reference string. In file and disk management, a category-specific allocation strategy (indexed for records, contiguous for lecture video) was paired with SCAN disk scheduling, chosen over the raw-faster SSTF specifically for its bounded-wait fairness guarantee under exam-period load, at a measured cost of 331 versus 252 cylinders of total head movement.

Every numeric result in this report — the Banker's safety trace, both CPU-scheduling Gantt charts and their metrics, the page-fault counts for FIFO/LRU/Optimal, and the head-movement totals for FCFS/SSTF/SCAN/C-SCAN — was produced by executable, reproducible Python simulations rather than manual calculation, and is presented in full in Appendix A. The constraints stated in the assignment brief were satisfied throughout: at least 8 processes and 4 resource types were modelled, the specified 16 GB/4 KB memory configuration and 200-cylinder/10+-request disk configuration were used exactly, and both a normal case and multiple adverse cases (an unsafe request, Belady's anomaly, an FCFS negative control, and a starvation-risk scenario) were validated. The principal limitation of this report is scale: the CPU and disk simulations use eight processes and twelve requests respectively, chosen to be large enough to be non-trivial and small enough to trace by hand for verification, rather than the full 500–800-process, sustained-load scale UniCore must eventually

handle; a natural extension is to re-run each simulator against synthetically generated traces at that larger scale to confirm the qualitative recommendations of Section 6 continue to hold quantitatively.

Student Reflection

Working through UniCore as one system rather than as three separate textbook exercises made clear how tightly these layers actually interact: the CPU-scheduling choice in Section 6.1 changes which processes are resident and active at any moment, which changes the working-set pressure the memory manager sees (Section 2, Q12), which in turn changes how many page faults — and therefore how much extra disk traffic — the disk scheduler in Section 6.3 has to absorb. A page-replacement or disk-scheduling algorithm that looks optimal in isolation (as SSTF does on raw head movement) can be the wrong choice once its knock-on effect on a different subsystem's fairness or predictability is considered, which only becomes visible when the subsystems are analysed together rather than in separate assignments.

Which design decision had the greatest impact on system performance, and why?

The choice of deadlock-handling strategy (avoidance via Banker's Algorithm, Section 6.4) had the largest impact, not on raw throughput, but on the system's worst-case correctness guarantee: it is the one decision in this report that converts a possible catastrophic failure mode (an undetected circular wait freezing examination processing) into a bounded, provably-avoided event at the cost of a small, easily-affordable per-request check. Every other decision in this report (scheduling policy, replacement algorithm, disk algorithm) trades off average-case performance metrics against each other, but only the deadlock strategy determines whether the system can enter an unrecoverable state at all.

What would we improve if the workload grew from 800 to 2,000 concurrent users?

- Re-validate the working-set/thrashing analysis (Q12) at 2,000-user scale, since the linear frame-budget argument used for 800 users may no longer hold once RAM pressure crosses a critical threshold; a dynamic, feedback-based frame-allocation policy would likely be needed rather than the current static per-class reservation.
- Move from a single centralized Banker's Algorithm check to a sharded or hierarchical resource-manager design, since a single global safety check across 2,000 processes' Need vectors would become a serialization bottleneck at the rate high-concurrency exam submission would demand.

- Re-benchmark SCAN against C-SCAN and against a weighted/priority-aware disk scheduler at a proportionally larger simultaneous-request count, since the 12-request workload used in this report may no longer represent realistic burst sizes at 2.5x the user count.
- Introduce a concurrency-safe, sharded variant of the writer-priority Readers–Writers protocol (e.g. per-examination-record locks rather than one global lock) so that unrelated students' record updates do not serialize behind each other at 2,000-user scale.

References

1. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed., Wiley, 2018.
2. M. J. Bach, The Design of the UNIX Operating System, Prentice-Hall, 1986.
3. W. Stallings, Operating Systems: Internals and Design Principles, 9th ed., Pearson, 2017.
4. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed., Pearson, 2014.
5. IEEE/The Open Group, POSIX.1-2017 (IEEE Std 1003.1-2017), "Portable Operating System Interface (POSIX)".
6. Python Software Foundation, "Python 3 Language and Standard Library Reference," https://docs.python.org/3/.
7. Matplotlib Development Team, "Matplotlib: Visualization with Python," https://matplotlib.org.
8. Project GitHub repository (source code, test cases, results, README): <insert group repository URL here before submission>.

Appendix A: Implementation / Source Code

A.1 Banker’s Algorithm — simulations.py

```
import itertools

print("="*70)
print("1. BANKER'S ALGORITHM")
print("="*70)

processes = ['P0','P1','P2','P3','P4','P5','P6','P7']
resources = ['DB','FL','PR','BK']  # Database-conn, File-lock, Print/report, Backup/IO-channel

Total = {'DB':9, 'FL':6, 'PR':5, 'BK':9}

Allocation = {
 'P0': {'DB':2,'FL':0,'PR':0,'BK':1},
 'P1': {'DB':1,'FL':1,'PR':0,'BK':0},
 'P2': {'DB':2,'FL':1,'PR':1,'BK':2},
 'P3': {'DB':0,'FL':1,'PR':1,'BK':1},
 'P4': {'DB':1,'FL':0,'PR':1,'BK':0},
 'P5': {'DB':0,'FL':1,'PR':0,'BK':2},
```

```
 'P6': {'DB':1,'FL':1,'PR':1,'BK':1},
 'P7': {'DB':0,'FL':0,'PR':0,'BK':1},
}

Max = {
 'P0': {'DB':5,'FL':1,'PR':1,'BK':2},
 'P1': {'DB':3,'FL':2,'PR':1,'BK':1},
 'P2': {'DB':6,'FL':2,'PR':2,'BK':3},
 'P3': {'DB':2,'FL':2,'PR':2,'BK':2},
 'P4': {'DB':4,'FL':1,'PR':2,'BK':1},
 'P5': {'DB':1,'FL':2,'PR':1,'BK':3},
 'P6': {'DB':3,'FL':2,'PR':2,'BK':2},
 'P7': {'DB':2,'FL':1,'PR':1,'BK':2},
}

def col_sum(d, key):
    return sum(d[p][key] for p in d)

Allocated_total = {r: col_sum(Allocation, r) for r in resources}
Available = {r: Total[r] - Allocated_total[r] for r in resources}

print("Total resources      :", Total)
print("Allocated (sum)      :", Allocated_total)
print("Available (Total-Alloc):", Available)

Need = {p: {r: Max[p][r]-Allocation[p][r] for r in resources} for p in processes}
print("\nNeed matrix:")
for p in processes:
    print(f"  {p}: {Need[p]}")
for p in processes:
    for r in resources:
        assert Need[p][r] >= 0, f"Max < Allocation for {p},{r}"
```

```
def safety_algorithm(Available, Need, Allocation, processes, resources, verbose=True):
    Work = dict(Available)
    Finish = {p: False for p in processes}
    safe_seq = []
    changed = True
    steps = []
    while True:
        progressed = False
        for p in processes:
            if not Finish[p] and all(Need[p][r] <= Work[r] for r in resources):
                for r in resources:
                    Work[r] += Allocation[p][r]
                Finish[p] = True
                safe_seq.append(p)
                steps.append((p, dict(Work)))
                progressed = True
        if not progressed:
            break
    ok = all(Finish.values())
    return ok, safe_seq, steps

ok, seq, steps = safety_algorithm(Available, Need, Allocation, processes, resources)
print("\nSafety algorithm result:", "SAFE" if ok else "UNSAFE")
print("Safe sequence:", seq)
print("\nStep-by-step Work vector after each process finishes:")
for p, w in steps:
    print(f"  after {p}: Work={w}")

print("\n--- Additional resource request that makes the system UNSAFE ---")
Request_P2 = {'DB':2,'FL':1,'PR':1,'BK':1}
print("Request by P2:", Request_P2)
feasible = all(Request_P2[r] <= Need['P2'][r] for r in resources) and all(Request_P2[r] <=
Available[r] for r in resources)
```

```
print("Request <= Need[P2]?", all(Request_P2[r] <= Need['P2'][r] for r in resources))
print("Request <= Available?", all(Request_P2[r] <= Available[r] for r in resources))

if feasible:
    Avail2 = {r: Available[r]-Request_P2[r] for r in resources}
    Alloc2 = {p: dict(Allocation[p]) for p in processes}
    Alloc2['P2'] = {r: Allocation['P2'][r]+Request_P2[r] for r in resources}
    Need2 = {p: dict(Need[p]) for p in processes}
    Need2['P2'] = {r: Need['P2'][r]-Request_P2[r] for r in resources}
    ok2, seq2, _ = safety_algorithm(Avail2, Need2, Alloc2, processes, resources)
    print("Tentative Available after grant:", Avail2)
    print("System state after granting request:", "SAFE" if ok2 else "UNSAFE")
    print("Safe sequence (if any):", seq2 if ok2 else "NONE - request must be DENIED, P2 must
WAIT")
else:
    print("Request denied immediately (exceeds Need or Available) per Banker's request algorithm
rule.")
```

Listing A.1 — Banker's Algorithm: matrices, safety algorithm, and the unsafe-request demonstration.

A.2 CPU Scheduling — cpu_sched.py

```
print("="*70)
print("2. CPU SCHEDULING")
print("="*70)

# 8 processes: interactive exam services (short bursts, high priority=low number)
# and background jobs (long bursts, low priority=high number)
procs = [
    # pid, arrival, burst, priority (1=highest), kind
    ("P0", 0, 4,  1, "Exam-submit"),
    ("P1", 1, 3,  1, "Exam-fetch"),
    ("P2", 2, 8,  3, "Library-index"),
    ("P3", 3, 2,  1, "Exam-autosave"),
    ("P4", 4, 10, 4, "Backup"),
    ("P5", 5, 6,  2, "Research-job"),
    ("P6", 6, 5,  2, "IoT-monitor"),
    ("P7", 7, 9,  4, "Backup-verify"),
]

def fmt(rows, headers):
    widths = [max(len(str(h)), max(len(str(r[i])) for r in rows)) for i,h in enumerate(headers)]
    line = " | ".join(str(h).ljust(widths[i]) for i,h in enumerate(headers))
    print(line); print("-"*len(line))
    for r in rows:
        print(" | ".join(str(r[i]).ljust(widths[i]) for i in range(len(headers))))

# ---------- Priority Scheduling (non-preemptive, lower number = higher priority) ----------
def priority_np(procs):
    procs = sorted(procs, key=lambda x: x[1])
    remaining = list(procs)
    t = 0
    gantt = []
    result = {}
    while remaining:
```

```
        ready = [p for p in remaining if p[1] <= t]
        if not ready:
            t = min(p[1] for p in remaining)
            ready = [p for p in remaining if p[1] <= t]
        nxt = min(ready, key=lambda x: (x[3], x[1]))
        start = max(t, nxt[1])
        finish = start + nxt[2]
        gantt.append((nxt[0], start, finish))
        result[nxt[0]] = {
            "arrival": nxt[1], "burst": nxt[2], "start": start, "finish": finish,
            "waiting": start - nxt[1], "turnaround": finish - nxt[1], "response": start - nxt[1]
        }
        t = finish
        remaining.remove(nxt)
    return gantt, result

# ---------- Round Robin (quantum = 4) ----------
def round_robin(procs, quantum=4):
    from collections import deque
    procs_sorted = sorted(procs, key=lambda x: x[1])
    remaining_bt = {p[0]: p[2] for p in procs_sorted}
    arrival = {p[0]: p[1] for p in procs_sorted}
    first_response = {}
    finish_time = {}
    q = deque()
    t = 0
    i = 0
    gantt = []
    n = len(procs_sorted)
    # push processes that arrive at time 0
    while i < n and procs_sorted[i][1] <= t:
        q.append(procs_sorted[i][0]); i += 1
    if not q and i < n:
```

```
    t = procs_sorted[i][1]
    while i < n and procs_sorted[i][1] <= t:
        q.append(procs_sorted[i][0]); i += 1
while q:
    pid = q.popleft()
    if pid not in first_response:
        first_response[pid] = t
    run = min(quantum, remaining_bt[pid])
    start = t
    t += run
    remaining_bt[pid] -= run
    gantt.append((pid, start, t))
    # enqueue any new arrivals during this run, BEFORE re-adding current if it still needs time
    while i < n and procs_sorted[i][1] <= t:
        q.append(procs_sorted[i][0]); i += 1
    if remaining_bt[pid] > 0:
        q.append(pid)
    else:
        finish_time[pid] = t
    if not q and i < n:
        t = max(t, procs_sorted[i][1])
        while i < n and procs_sorted[i][1] <= t:
            q.append(procs_sorted[i][0]); i += 1
result = {}
for p in procs_sorted:
    pid = p[0]
    result[pid] = {
        "arrival": p[1], "burst": p[2],
        "finish": finish_time[pid],
        "waiting": finish_time[pid] - p[1] - p[2],
        "turnaround": finish_time[pid] - p[1],
        "response": first_response[pid] - p[1],
    }
```

```
    return gantt, result

print("\n--- Priority Scheduling (non-preemptive) ---")
g1, r1 = priority_np(procs)
print("Gantt sequence:", " -> ".join(f"{p}[{s}-{f}]" for p,s,f in g1))
rows=[]
for p in procs:
    pid=p[0]; d=r1[pid]

rows.append([pid,d['arrival'],d['burst'],d['start'],d['finish'],d['waiting'],d['turnaround'],d['response']])
fmt(rows, ["PID","Arr","Burst","Start","Finish","Waiting","Turnaround","Response"])
avg_w1 = sum(r1[p[0]]['waiting'] for p in procs)/len(procs)
avg_t1 = sum(r1[p[0]]['turnaround'] for p in procs)/len(procs)
avg_r1 = sum(r1[p[0]]['response'] for p in procs)/len(procs)
print(f"Average waiting={avg_w1:.2f}  turnaround={avg_t1:.2f}  response={avg_r1:.2f}")

print("\n--- Round Robin (quantum = 4) ---")
g2, r2 = round_robin(procs, 4)
print("Gantt sequence:", " -> ".join(f"{p}[{s}-{f}]" for p,s,f in g2))
rows=[]
for p in procs:
    pid=p[0]; d=r2[pid]
    rows.append([pid,d['arrival'],d['burst'],d['finish'],d['waiting'],d['turnaround'],d['response']])
fmt(rows, ["PID","Arr","Burst","Finish","Waiting","Turnaround","Response"])
avg_w2 = sum(r2[p[0]]['waiting'] for p in procs)/len(procs)
avg_t2 = sum(r2[p[0]]['turnaround'] for p in procs)/len(procs)
avg_r2 = sum(r2[p[0]]['response'] for p in procs)/len(procs)
print(f"Average waiting={avg_w2:.2f}  turnaround={avg_t2:.2f}  response={avg_r2:.2f}")

print("\n--- Interactive (P0,P1,P3 = exam) vs Background (P4,P7 = backup) response under each ---
")
for name, r in [("Priority-NP", r1), ("Round Robin q=4", r2)]:
    exam_resp = [r[p][ 'response'] for p in ['P0','P1','P3']]
```

```
    backup_resp = [r[p]['response'] for p in ['P4','P7']]
    print(f"{name}: exam avg response={sum(exam_resp)/len(exam_resp):.2f}  backup avg
response={sum(backup_resp)/len(backup_resp):.2f}")
```

Listing A.2 — Priority (non-preemptive) and Round Robin (quantum=4) schedulers.

A.3 Memory Management — memory.py

```
print("="*70)
print("3. MEMORY MANAGEMENT")
print("="*70)

RAM = 16 * 1024**3        # 16 GB in bytes
PAGE = 4 * 1024           # 4 KB
frames = RAM // PAGE
print(f"Physical RAM = 16 GB = {RAM} bytes; Page size = 4 KB = {PAGE} bytes")
print(f"Number of physical frames = RAM / PAGE = {frames:,}")

LOGICAL = 64 * 1024**2      # 64 MB
pages = LOGICAL // PAGE
print(f"\nLogical address space = 64 MB = {LOGICAL} bytes")
print(f"Number of pages = {LOGICAL} / {PAGE} = {pages:,} pages")
```

```
import math
offset_bits = int(math.log2(PAGE))
page_bits = int(math.log2(pages))
frame_bits = int(math.log2(frames))
print(f"\nOffset bits (log2 4096) = {offset_bits}")
print(f"Page-number bits for this 64MB process (log2 {pages}) = {page_bits}")
print(f"Frame-number bits needed to address {frames:,} physical frames (log2) = {frame_bits}")
pte_size = 4  # bytes, typical (20-bit frame#, plus valid/dirty/ref/protection bits, rounded to 4B)
pt_size_single_process = pages * pte_size
print(f"\nSingle-level page table size for this 64MB process = {pages:,} entries x {pte_size} bytes =
{pt_size_single_process:,} bytes = {pt_size_single_process/1024:.1f} KB")

print("""
For a FULL 64-bit-style flat page table covering the entire 4M-frame physical
address space per process, a single-level table would need frame_bits=22 bits
per entry region and, more importantly, if many of the 500-800 concurrent
UniCore processes each had a flat table sized to the full logical space, the
memory spent on page tables alone could become significant. This motivates
using a multi-level (hierarchical) or inverted page table in the design
(see Section: Memory-Management Design).
""")

print("="*70)
print("4. PAGE REPLACEMENT: FIFO vs LRU vs OPTIMAL")
print("="*70)

# reference string: 16 references (exam PDF pages + library pages cycling)
ref = [7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0]
print("Reference string (16 refs):", ref)

def fifo(ref, nframes):
    frames = []
    faults = 0
```

```
    trace = []
    for r in ref:
        if r not in frames:
            faults += 1
            if len(frames) >= nframes:
                frames.pop(0)
            frames.append(r)
            trace.append((r, list(frames), True))
        else:
            trace.append((r, list(frames), False))
    return faults, trace

def lru(ref, nframes):
    frames = []
    faults = 0
    trace = []
    recency = {}
    for idx, r in enumerate(ref):
        if r not in frames:
            faults += 1
            if len(frames) >= nframes:
                lru_page = min(frames, key=lambda p: recency[p])
                frames.remove(lru_page)
            frames.append(r)
            trace.append((r, list(frames), True))
        else:
            trace.append((r, list(frames), False))
        recency[r] = idx
    return faults, trace

def optimal(ref, nframes):
    frames = []
    faults = 0
```

```
    trace = []
    for idx, r in enumerate(ref):
        if r not in frames:
            faults += 1
            if len(frames) >= nframes:
                future = ref[idx+1:]
                def next_use(p):
                    return future.index(p) if p in future else float('inf')
                victim = max(frames, key=next_use)
                frames.remove(victim)
            frames.append(r)
            trace.append((r, list(frames), True))
        else:
            trace.append((r, list(frames), False))
    return faults, trace

for nf in (3, 4):
    print(f"\n--- Frames = {nf} ---")
    for name, fn in [("FIFO", fifo), ("LRU", lru), ("Optimal", optimal)]:
        faults, trace = fn(ref, nf)
        seq = "".join("F" if t[2] else "." for t in trace)
        print(f"{name:8s}: faults={faults:2d}  fault-pattern={seq}")

print("\n--- Belady's anomaly check: FIFO faults as frames increase ---")
for nf in range(2,7):
    faults,_ = fifo(ref, nf)
    print(f"  FIFO frames={nf}: faults={faults}")
for nf in range(2,7):
    faults,_ = lru(ref, nf)
    print(f"  LRU  frames={nf}: faults={faults}")
```

Listing A.3 — Frame/page arithmetic and FIFO/LRU/Optimal page-replacement simulation.

A.4 Disk Scheduling — disk_sched.py

```
print("="*70)
print("5. DISK SCHEDULING")
print("="*70)

cylinders = 200
head_start = 53
requests = [98, 183, 37, 122, 14, 124, 65, 67, 15, 130, 200-1, 143]  # 12 requests
print(f"Disk has {cylinders} cylinders (0..{cylinders-1}); initial head position = {head_start}")
print("Pending requests:", requests)

def fcfs(start, reqs):
    seq = [start] + reqs
    total = sum(abs(seq[i+1]-seq[i]) for i in range(len(seq)-1))
    return reqs, total

def sstf(start, reqs):
    reqs = list(reqs)
    seq = []
```

```
    cur = start
    total = 0
    while reqs:
        nxt = min(reqs, key=lambda r: abs(r-cur))
        total += abs(nxt-cur)
        cur = nxt
        seq.append(cur)
        reqs.remove(nxt)
    return seq, total

def scan(start, reqs, max_cyl, direction="up"):
    reqs = sorted(reqs)
    seq = []
    total = 0
    cur = start
    if direction == "up":
        upper = [r for r in reqs if r >= start]
        lower = [r for r in reqs if r < start]
        path = list(upper)
        if not upper or upper[-1] != max_cyl-1:
            path.append(max_cyl-1)
        path += list(reversed(lower))
    else:
        lower = [r for r in reqs if r <= start]
        upper = [r for r in reqs if r > start]
        path = list(reversed(lower))
        if not lower or lower[0] != 0:
            path.append(0)
        path += list(reversed(upper))
    for p in path:
        total += abs(p-cur)
        cur = p
        seq.append(p)
```

```
    return seq, total

def cscan(start, reqs, max_cyl):
    reqs = sorted(reqs)
    seq=[]
    total=0
    cur=start
    upper=[r for r in reqs if r>=start]
    lower=[r for r in reqs if r<start]
    path = list(upper)
    if not upper or upper[-1] != max_cyl-1:
        path.append(max_cyl-1)
    path.append(0)
    path += lower
    for p in path:
        total += abs(p-cur)
        cur = p
        seq.append(p)
    return seq, total

seqF, totF = fcfs(head_start, requests)
print(f"\nFCFS  order = {[head_start]+seqF}")
print(f"FCFS total head movement = {totF} cylinders")

seqS, totS = sstf(head_start, requests)
print(f"\nSSTF  order = {[head_start]+seqS}")
print(f"SSTF total head movement = {totS} cylinders")

seqSc, totSc = scan(head_start, requests, cylinders, "up")
print(f"\nSCAN(up, then reverses at {cylinders-1}) order = {[head_start]+seqSc}")
print(f"SCAN total head movement = {totSc} cylinders")

seqC, totC = cscan(head_start, requests, cylinders)
```

```
print(f"\nC-SCAN order = {[head_start]+seqC}")
print(f"C-SCAN total head movement = {totC} cylinders")


print("\nSummary:")
for name, tot in [("FCFS", totF), ("SSTF", totS), ("SCAN", totSc), ("C-SCAN", totC)]:
    print(f"  {name:8s}: {tot} cylinders  (avg seek/request = {tot/len(requests):.2f})")
```

Listing A.4 — FCFS/SSTF/SCAN/C-SCAN total head-movement computation.